第5章 树和二叉树

树与二叉树
——非线性数据结构，是以分支关系定义的层次结构。

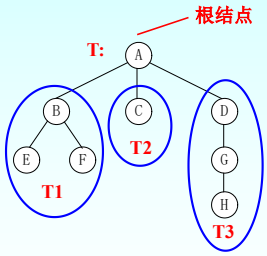
5.1 树的定义

5.2 二叉树

5.3 树和森林

5.4 哈夫曼树与哈夫曼编码

5.1 树的定义



树（Tree）：是由n(n≥0)个结点组成的有限集合T。当n＝0时为空树；在任一一棵非空树中：

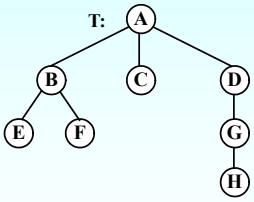
1)有且仅有一个特定的结点——根结点(root)；

2)其余结点分成m(m≥0)个互不相交的有限集T1,T2,...,Tm。其中每个集合Ti都是一棵树，称T1,T2,...,Tm为根结点的子树（subtree）。

定义递归：一棵树是由子树构成的，而子树又是由更小的子树构成的。最小的树是由一个结点构成的。

例：结点集合T＝{A,B,C,D,E,F,G,H}，其中A为根结点，其余的结点构成互不相交的集合T1={B,E,F},T2={C},T3={D,G,H}为根结点的三个子树，同时其本身也是一棵树。

树的相关术语



•结点(node)：表示树中的元素。

•结点的度(degree)：结点拥有的子树数。
例：A的度为3，B的度为2，C的度为0，D的度为1。

•树的度：一棵树中最大的结点度数。
例：树的度为3。

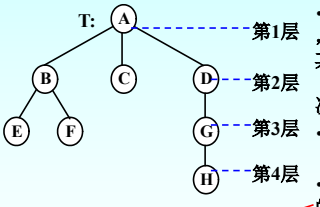
•叶子(leaf)：度为零的结点。
例：C、E、F、H都是叶子。

•孩子(child)：一个结点的后继结点，称为该结点的孩子。
例：B、C、D是A的孩子。

•双亲(parents)：一个结点的前趋结点，称为该结点的双亲。
例：A是B、C、D的双亲。

•兄弟(sibling)：同一双亲的孩子，互称兄弟。
例：B、C、D互为兄弟。

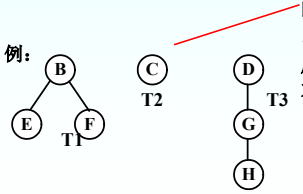
树的相关术语（续）



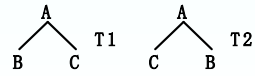
•结点的层次(level)：从根结点开始计算，根为第一层，根的直接后继为第二层，其余各层依次类推。
例：C结点的层次为2，A结点的层次为1。

•深度(depth)：树中结点的最大层次数。
例：树的深度为4。

•森林(forest)：是m(m≥0)棵互不相交的树的集合。



•有序树：树中结点同层内从左到右有次序排列，不能互换的称为有序树。反之称为无序树。



若为有序树，则T1≠T2
若为无序树，则T1＝T2

5.2 二叉树

5.2.1 二叉树的定义

5.2.2 二叉树的性质

5.2.3 二叉树的存储结构

5.2.4 二叉树的遍历

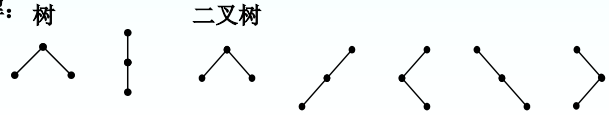
5.2.1 二叉树的定义

定义：
二叉树是n(n≥0)个结点的有限集，它或为空树(n=0)，或由一个根结点和两棵分别称之为左子树和右子树的互不相交的二叉树构成。

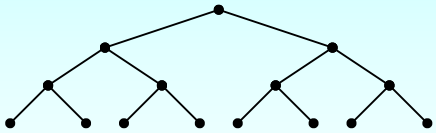
二叉树的特点：
•二叉树的每个结点至多有两个孩子，即不存在度大于2的结点。
•二叉树的子树有明确的左、右之分，不能随意变换。

例：由3个结点构成多少个不同形态的树与二叉树？

解：树　　二叉树



5.2.2 二叉树的性质



➢性质1　二叉树的第i层上最多含有2^{i-1}(i≥1)个结点。

证：（归纳法）

i=1，则结点数为$2^{1-1}=1$，为根结点；

若已知第i-1层上至多有$2^{(i-1)-1}=2^{i-2}$个结点，由于二叉树每个结点的度数最大为2，因此，第i层上的结点数最多为第i－1层上的结点数的2倍，即

$2\times 2^{i-2}=2^{i-1}$　（证毕）

二叉树的性质2

性质2　深度为h的二叉树最多有2^h-1个结点(h≥1)。

证：由性质1可得，在深度为h的二叉树中至多含有结点数为第1层到第h层上最大结点数之和，即

$$\sum_{i=1}^{h}(\text{第}i\text{层上结点最大数})=\sum_{i=1}^{h}2^{i-1}=2^h-1$$　（证毕）

二叉树的性质3

性质3　若在任何一棵二叉树中，有n_0个叶子结点，有n_2个度为2的结点，则必有$n_0=n_2+1$。

证：设n1是度为1的结点数。因为二叉树所有结点的度数均不大于2，则二叉树总结点数为

$n=n_0+n_1+n_2$.....(1)

二叉树中除根结点外，其它结点都有一个指针与其双亲相连，若指针数为b，则有

$b=n-1$ .....(2)

而这些指针又可以看作是度为1和度为2的结点与它们的孩子之间的联系，则

$b=n_1+2n_2$.....(3)

由（2）和（3），得

$n=n_1+2n_2+1$.....(4)

比较（1）和（4），得

$n_0=n_2+1$　（证毕）

二叉树性质的例题

例：有24个结点的二叉树，其中度为1的结点有9个，则度为2的结点有_____。

解：根据已知条件、性质3和二叉树的特点，得

$n_0+n_1+n_2=24$

$n_1=9$

$n_0=n_2+1$

解得　$n_2=7$

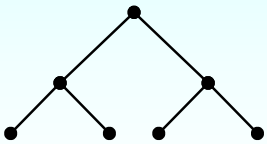
特殊形式的二叉树

(1)满二叉树

➢定义：

深度为h且含有2^h-1个结点的二叉树为满二叉树。

例：试构造深度为3的满二叉树。



➢特点：

- 每一层上的结点数都达到最大值。
- 不存在度数为1的结点。
- 每个分支结点均有两棵深度相同的子树，且叶子结点都在最下一层。

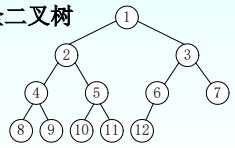
(2)完全二叉树

➢定义：

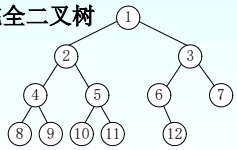
若一棵二叉树除最下层外、其它层构成一满二叉树，且最下层中的结点都集中在该层最左边的若干位置上，则此二叉树称为完全二叉树。

例：判断下列两棵二叉树是否为完全二叉树。

完全二叉树



不完全二叉树



➢特点：

- 满二叉树一定是完全二叉树，但完全二叉树不一定是满二叉树。
- 在完全二叉树中，若某个结点没有左孩子，则它一定没有右孩子，即该结点必为叶子结点。

二叉树的性质4

性质4　具有n个结点的完全二叉树的深度为$\lfloor \log_2 n \rfloor + 1$。

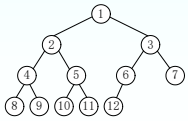
证：设所求完全二叉树的深度为h，由完全二叉树定义可知，其前h－1层是深度为h－1的满二叉树，具有$2^{h-1}-1$个结点。另一方面，由性质2，得$n \leqslant 2^h - 1$，即

$$2^{h-1} - 1 < n \leqslant 2^h - 1$$

或　$2^{h-1} \leqslant n < 2^h$

得　$h - 1 \leqslant \log_2 n < h$

因为h为整数，所以 $h = \lfloor \log_2 n \rfloor + 1$。　（证毕）



验证：上图中的完全二叉树，有n＝12个结点，深度h=3+1=4。

完全二叉树的例题

例1：深度为k的完全二叉树至少有2^{k-1}个结点，至多有个2^k-1结点。若按自上而下、从左到右次序给结点编号（从1开始），则编号最小的叶子结点的编号是$2^{k-2}+1$。

例2：已知一棵完全二叉树的第6层有8个叶子结点，则该完全二叉树的结点个数最多是_____。

A. 39　　B. 52　　C. 111　　D. 119

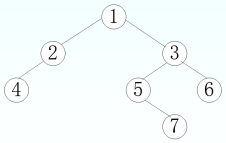
(3)平衡二叉树

➢定义：

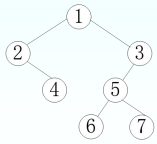
平衡二叉树又称AVL树，或是一棵空树，或是具有下列性质的二叉树：它的左、右子树都是平衡二叉树，且左子树和右子树的深度之差的绝对值不超过1。

结点的平衡因子：结点的左子树深度减去其右子树深度，可能为－1，0和1。

例：判断下列两棵二叉树是否为平衡二叉树。



平衡二叉树

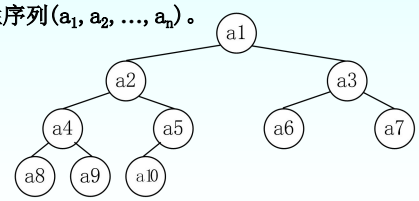


非平衡二叉树

5.2.3 二叉树的存储结构

(1)顺序存储结构

对于具有n个结点的完全二叉树，从根开始，自上层到下层，每层自左至右地给所有结点编号，就能得到一个足以反映二叉树的线性序列($a_1, a_2, \ldots, a_n$)。



a1	a2	a3	a4	a5	a6	a7	a8	a9	a10

二叉树的性质5

性质5：如果对一棵有n个结点的完全二叉树的结点按自上到下、自左到右顺序连续编号，则对任一结点a_i(1≤i≤n)有

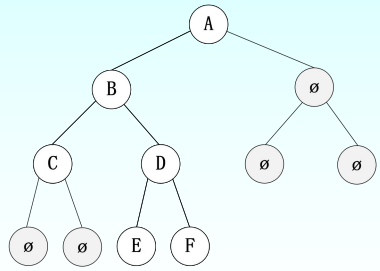
(1)如果i=1，则结点a_i是二叉树的根，无双亲结点;如果i>1，则其双亲结点是$a_{i/2}$ 。

(2)如果2i>n，则结点a_i无左孩子（结点i为叶子结点）;否则其左孩子为结点a_{2i}。

(3)如果2i+1>n，则结点a_i无右孩子；否则其右孩子是结点a_{2i+1}。

非完全二叉树的顺序存储

- 增加虚结点将非完全二叉树补充成完全二叉树。



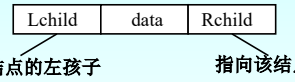
顺序存储结构：

A	B	Φ	C	D	Φ	Φ	Φ	Φ	E	F

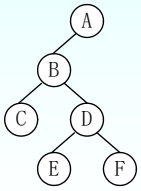
- 最坏情况下，深度为k且只有k个结点的单支树需要长度为2^k-1的存储单元。

(2)链式存储结构

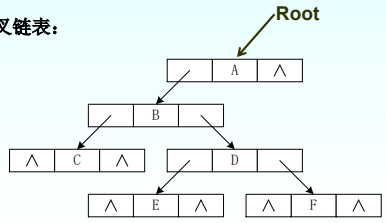
二叉链表：二叉树中每个结点的结构：



例：



二叉链表：



(1) 一个二叉链表由根指针root唯一确定。若二叉树为空，则root=NULL；若结点的某个孩子不存在，则相应的指针为空。
(2) 具有n个结点的二叉链表，一共有2n个指针域。其中只有n-1个用来指示结点的左、右孩子，其余的n+1个指针域为空。

例题：

有10个结点的二叉树的存储结构如下，其中i是结点序号，Lchild[i]和Rchild[i]分别是结点i的左、右孩子指针域，data[i]是结点i的数据域。根结点是序号为6的结点。

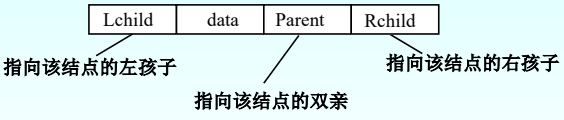
（1）画出该二叉树的逻辑结构。

（2）写出该二叉树的顺序存储结构。

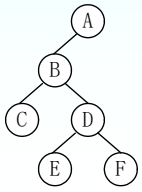
i	1	2	3	4	5	6	7	8	9	10
Lchild	Λ	Λ	2	3	7	5	8	Λ	10	1
Data	j	h	f	d	b	a	c	e	g	i
Rchild	Λ	Λ	Λ	9	4	Λ	Λ	Λ	Λ	Λ

三叉链表

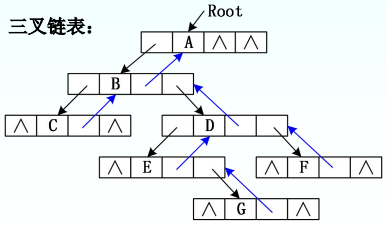
为了便于找到结点的双亲，则可在结点结构中增加一个指向其双亲结点的指针域。即每个结点有三个指针域：



例：



三叉链表：



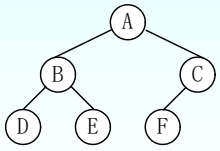
5.2.4 二叉树的遍历

遍历：按照某条搜索路线，依次访问某数据结构中的全部结点，而每个结点只能被访问一次。

（1）先序遍历（DLR）
- 首先访问根结点
- 然后按先序遍历根结点的左子树
- 最后按先序遍历根结点的右子树

（2）中序遍历（LDR）
- 首先按中序遍历根结点的左子树
- 然后访问根结点
- 最后按中序遍历根结点的右子树

（3）后序遍历（LRD）
- 首先按后序遍历根结点的左子树
- 然后按后序遍历根结点的右子树
- 最后访问根结点



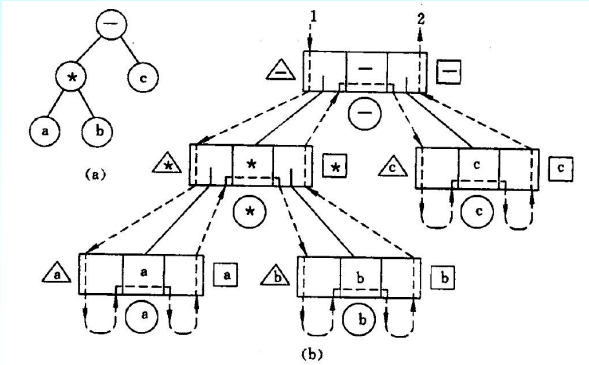
先序遍历的递归算法

```
void  PreorderTraverse(BTNode  *T)
  { if (T!=NULL);     /* 如果树空，返回*/
        {   visit(T->data);     /* 访问根结点 */
            PreorderTraverse(T->Lchild); /* 先序遍历左子树 */
            PreorderTraverse(T->Rchild); /* 先序遍历右子树 */

        }
  }
```

思考：中序遍历、后序遍历的递归算法？

三种遍历过程



从递归执行过程的角度来看，先序、中序、后序遍历是完全相同的（△○□分别表示先中后序遍历过程中访问到的结点信息输出）

先序遍历的非递归算法

若二叉树T为空，则返回；否则，令p=T；

⑴ 访问p所指向的结点；

⑵ q=p->Rchild ，若q不为空，则q进栈；

⑶ p=p->Lchild ，若p不为空，转(1)，否则转(4)；

⑷ 退栈到p ，转(1)，直到栈空为止。

```
#define MAX_NODE 50
void PreorderTraverse( BTNode *T)
    {   BTNode *Stack[MAX_NODE] ,*p=T, *q ;
        int top=0 ;
        if (T==NULL) printf(" Binary Tree is Empty!\n") ;
        else { do
                    { visit( p-> data ) ;  q=p->Rchild ;
                      if ( q!=NULL ) stack[++top]=q ;
                      p=p->Lchild ;
                      if (p==NULL) { p=stack[top] ;  top-- ; }
                    }
                  while (p!=NULL) ;
                }
    }
```

二叉树遍历的例题

例：已知先序、中序，求其后序。 (DLR、LDR→LRD)

先序：A、B、C、D、E、F、G、H、I

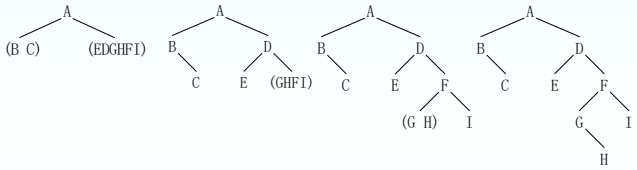
中序：B、C、A、E、D、G、H、F、I

分析：

①由先序得：A为根；再由中序知，A之前为A的左子树，A之后为A的右子树。

②再以B为根，C为B的右子树。

③以D为根，E为D的左子树，GHFI为D的右子树，依次类推。



二叉树遍历

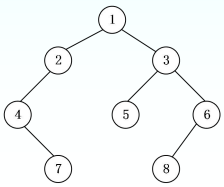
- 已知中序和先序、中序和后序，可以唯一确定二叉树！
- 先序和后序不能确定唯一的二叉树！

例：DLR为 1 2 4 7 3 5 6 8

LRD为 7 4 2 5 8 6 3 1

能否画出唯一的二叉树？

加条件：LDR为 4 _ _ 1 _ _ 8 _ ，能否画出唯一的二叉树？



二叉树遍历的例题

例：已知二叉树的先序遍历、中序遍历和后序遍历如下，其中各有一部分未给出其值。要求画出该二叉树。

先序遍历：__ __ G __ I __ D __ E；

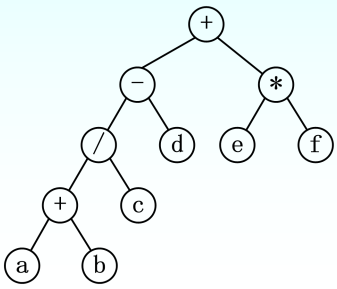
中序遍历：__ C __ F A __ H B __；

后序遍历：G __ __ C __ D __ B A。

二叉树遍历的应用

用二叉树描述表达式： 操作数作为二叉树的叶子结点;两个操作数之间的操作符作为二叉树的对应叶子结点的双亲结点。

例：表达式（a+b）/c-d+e*f



前序遍历：+-/+abcd*ef

前缀表示

中序遍历：(a+b)/c-d+e*f

中缀表示

后序遍历：ab+c/d-ef*+

后缀表示

5.3 树和森林

5.3.1 树的存储结构

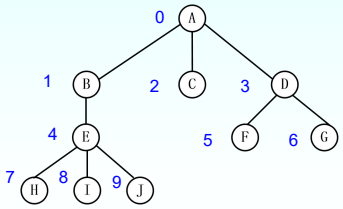
5.3.2 森林、树、二叉树的相互转化

5.3.3 树和森林的遍历

5.3.1 树的存储结构

1. 双亲表示法——利用每个结点（根结点除外）只有唯一的双亲的性质。

以一组连续空间存储树的结点，同时在每个结点中附设一个指示器指示其双亲结点的位置。



数组下标	结点值	结点指示器
0	A	−1
1	B	0
2	C	0
3	D	0
4	E	1
5	F	3
6	G	3
7	H	4
8	I	4
9	J	4

树的存储结构

2. 孩子表示法——树中每个结点可能有多棵子树，则可用多重链表，即每个结点有多个指针域，其中每个指针指向一棵子树的根结点。

（1）每个结点的指针域的个数相同，为树的度n。

data	child1	child2	……	childn

链表中会有很多空链域，空间较浪费

（2）每个结点的指针域的个数不同，为该结点的度d。

data	degree	child1	child2	……	childd

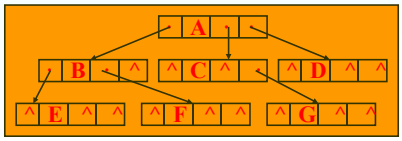
节约存储，但操作不方便

树的存储结构的例题

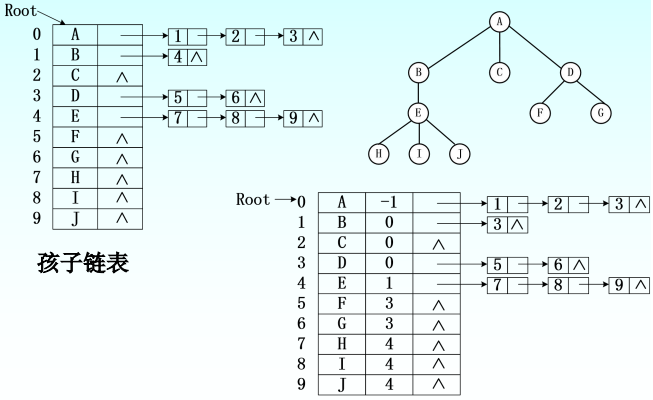
- 若树度为k, k=1为线性表，k=2时空链域最低，即二叉树。

例：一棵具有n个结点的k叉树，采用同构型存储结构，问有多少个空链域？

解：总链域=nk；非空链域=n-1；所以，空链域=n(k-1)+1。



树的存储结构



孩子链表

带双亲的孩子链表

树的存储结构

3. 孩子兄弟表示法——即二叉树表示法，以二叉树作为树的存储结构。二叉树中的结点的两个指针域分别指向树中对应结点的第一个孩子和下一个兄弟。

关键点：树怎么转换成二叉树？？

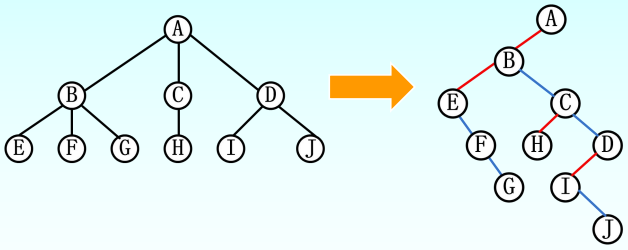
5.3.2 树、森林与二叉树之间的转换

1. 树转换为二叉树

树中的每个结点可能有多个孩子和多个兄弟 → 二叉树中的每个结点最多只能有2个孩子和1个兄弟

➢转换方法：
- 树的根作为二叉树的根；
- 某结点的最左孩子作为二叉树中对应结点的左孩子；
- 某结点的相邻右兄弟作为二叉树中对应结点的右孩子。

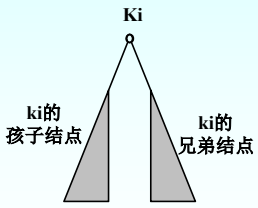
树转换为二叉树

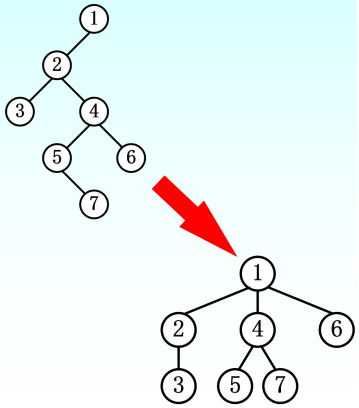


•二叉树中所有的左子树中的结点：双亲结点与第一个孩子的关系；且结点的孩子都在该结点的左子树中。

•二叉树中所有的右子树中的结点：相邻兄弟关系；且结点的兄弟及兄弟的孩子都在该结点的右子树中。

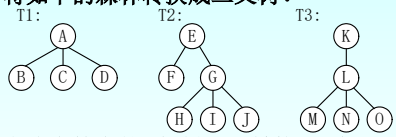
2. 二叉树转换为一般树



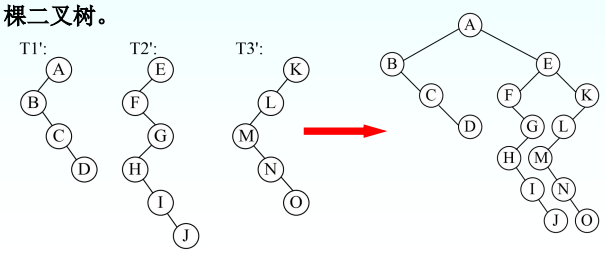


3. 森林转换为二叉树

例：将如下的森林转换成二叉树？

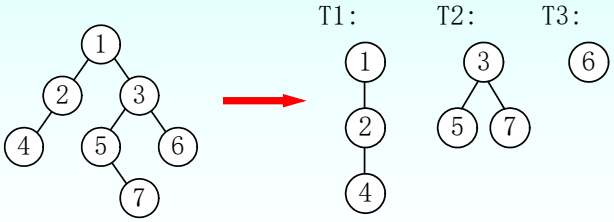


解：先将森林中的m棵树分别转换成m棵二叉树，再合并一棵二叉树。



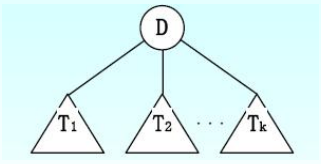
4. 二叉树转换为森林

例：将如下的二叉树转换成森林？



5.3.3 树和森林的遍历

树的遍历——在树T中，结点D是根，根的子树从左到右依次为T1，T2，…，Tk。

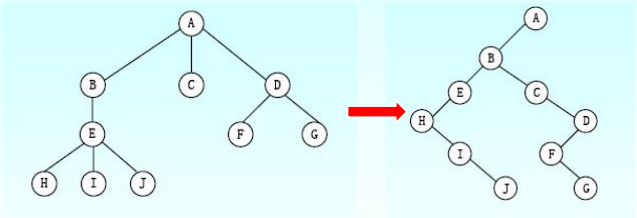


➢ 树T的先序遍历：若树T非空，则①访问根结点D； ②依次先序遍历根R的各子树T1，T2，…，Tk。

➢ 树T的后序遍历：若树T非空，则①依次后序遍历根T的各子树T1，T2，…，Tk； ②访问根结点D。

树的遍历的例题

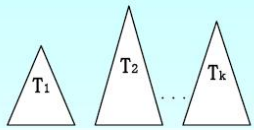
例：树的先序和后序遍历



① 先序遍历一棵树恰好等价于先序遍历该树对应的二叉树。

② 后序遍历树恰好等价于中序遍历该树对应的二叉树。

森林的遍历

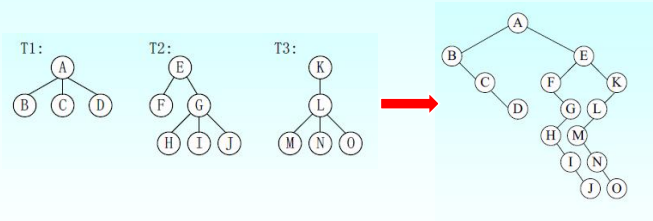


➢ 先序遍历森林： 若森林非空，则
①访问森林中第一棵树的根结点；
②先序遍历第一棵树中根结点的各子树所构成的森林；
③先序遍历除第一棵树外其它树构成的森林。

➢后序遍历森林： 若森林非空，则
①后序遍历森林中第一棵树的根结点的各子树所构成的森林；
②访问第一棵树的根结点；
③后序遍历除第一棵树外其它树构成的森林。

森林的遍历的例题

例：森林的先序和后序遍历



① 先序遍历森林等同于先序遍历该森林对应的二叉树。

② 后序遍历森林等同于中序遍历该森林对应的二叉树。

5.4 哈夫曼树和哈夫曼编码

5.4.1 哈夫曼树

5.4.2 哈夫曼编码

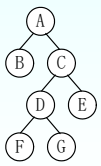
5.4.1 哈夫曼树

(1)基本术语

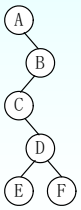
•路径：从根结点到某结点所经过的结点序列。

•路径长度：路径上经过的分支数目（等于路径上的结点数减1）。

•树的路径长度：从树的根到每一个结点的路径长度之和，记作PL。



PL＝0＋1×2＋2×2＋3×2＝12



PL＝0＋1＋2＋3＋4×2＝14

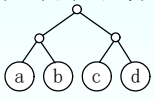
基本术语（续）

•二叉树的带权路径长度：若二叉树有m个叶子结点，每个叶子结点i分别赋予一个权值（w_i），则该二叉树的带权路径长度为

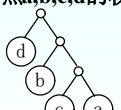
$$WPL = \sum_{i=1}^{m} w_i p_i$$

其中，pi为第i个叶子结点的路径长度。

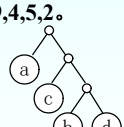
例：下列二叉树中叶子结点a,b,c,d的权分别是9,4,5,2。



WPL_1=9*2+4*2+5*2+2*2=40



WPL_2=2*1+4*2+5*3+9*3=52



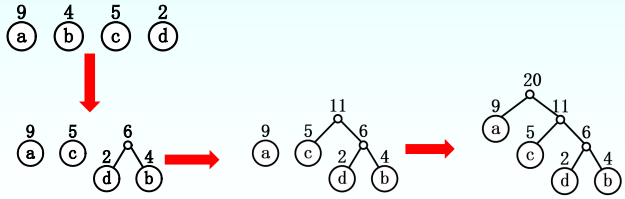
WPL_3=9*1+5*2+4*3+2*3=37

结论：满二叉树或完全二叉树的WPL不一定最小；
权值越大的结点离根越近，WPL越小。

（2）构造哈夫曼树

•哈夫曼树：又称为最优二叉树，是指给定一组权值，构造出的具有最小带权路径长度的二叉树。

例：已知4个叶子结点（a,b,c,d）分别带权（9,4,5,2）。求：1）构造哈夫曼树；2）该哈夫曼树的带权路径长度。



WPL=9*1+5*2+2*3+4*3=37

哈夫曼树的特点

•特点：

1）给定一组权值，构造出来的哈夫曼树不一定是唯一的；

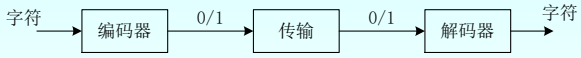
2）在哈夫曼树中，权值越大的叶子结点离根结点越近，而权值越小的结点离根结点越远；

3）哈夫曼树中没有度为1的结点。

例：若某Huffman树有9个叶子结点，这棵Huffman共有多少结点？

5.4.2 哈夫曼编码

•编码的作用：



•编码的分类：

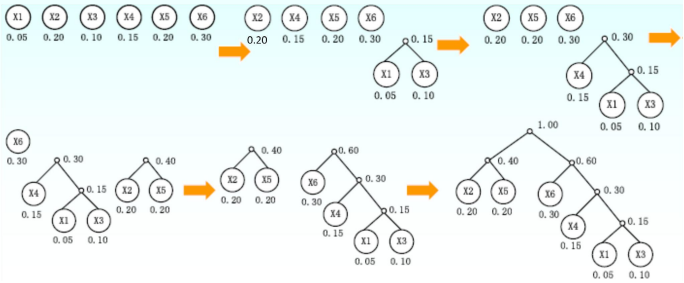
•等长编码——对所有字符分配相同位数的码长。

•不等长编码——出现频率高的字符具有较短的码长，出现频率低的字符具有较长的码长。

哈夫曼编码的例题

例：已知通信系统信源X为{ x1，x2，x3，x4，x5，x6 }，每信源出现的概率为{0.05，0.20，0.10，0.15，0.20，0.30}。
求：(1)画出Huffman码树；(2)Huffman编码；(3)平均码长；(4)与等长的自然编码相比，Huffman编码的压缩比是多少？

解：(1)Huffman树——每次选择权值最小的两个结点相结合，生成新的结点

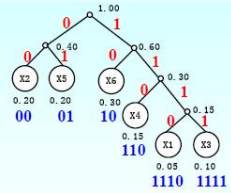


哈夫曼编码的例题

(2)Huffman码

①约定左分支表示“0”，右分支表示“1”

②从根到叶子结点的路径上的0和1组合作为该叶子结点的Huffman码



X1	X2	X3	X4	X5	X6
1110	00	1111	110	01	10

(3)平均码长——即WPL

$$WPL=\sum w_i p_i = 2\times(0.3+0.2+0.2)+3\times 0.15+4\times(0.1+0.05)=2.45$$

(4)压缩比

$$\text{压缩比}=\frac{\text{等长码平均码长}}{\text{不等长码平均码长}}=\frac{3}{2.45}=1.22$$

哈夫曼编码的习题

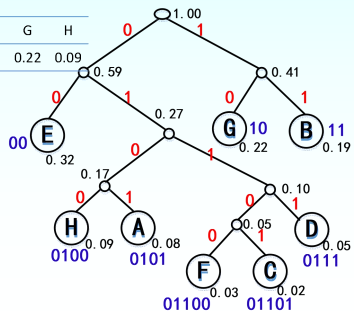
假设用于通信的电文有8个字母组成，字母在电文中出现的频率分别为：

A	B	C	D	E	F	G	H
8	19	2	5	32	3	22	9

试为这8个字母设计Huffman编码。

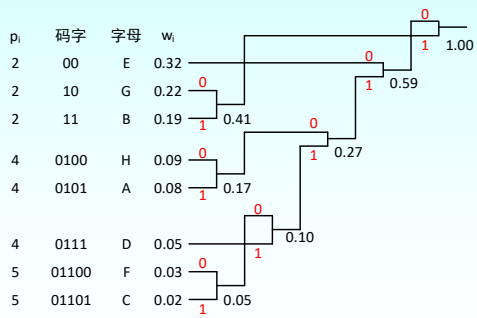
Huffman树入手的解法：

字母	A	B	C	D	E	F	G	H
w_i	0.08	0.19	0.02	0.05	0.32	0.03	0.22	0.09



哈夫曼编码的习题

严谨的解法：



$$WPL=\sum w_i p_i=(0.32+0.22+0.19)*2+(0.09+0.08+0.05)*4+(0.03+0.02)*5=2.59$$

$$\text{压缩比}=\frac{\text{等长码长}}{\text{不等长码长}}=\frac{3}{2.59}\approx 1.158$$